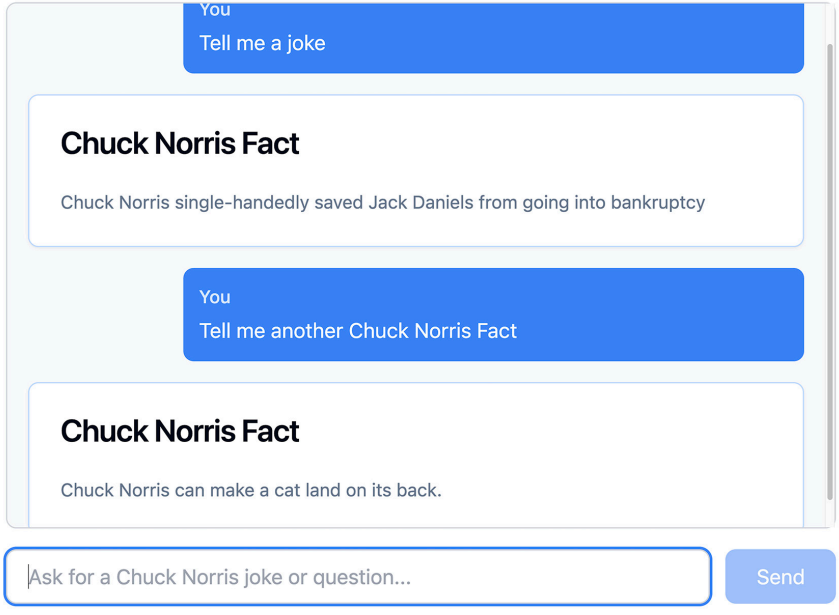


Figure 12.8 The user asks the chat interface for some Chuck Norris facts, and the request flows through the Next.js API to the LangChain agent. The agent uses its MCP client to call the Chuck Norris server via `stdio`, retrieving a fact from `api.chucknorris.io` that streams back to update the UI.

The chat interface demonstrates the MCP architecture in action. When a user requests a joke, the message flows from the application through the API route to the LangChain agent. The agent's MCP client communicates with the Chuck Norris MCP server via `stdio` transport, which calls https://api.chucknorris.io/ to retrieve a random joke or a joke from a specific category. Users can ask for a joke about movies or request a list of available categories. The response then streams back through the same components to update the user interface in real time, as shown in the screenshot.

12.6 The future of the MCP: Gateways, directories, and MCP-as-a-service

The MCP is still in its early stages, but adoption is accelerating. Developers, platform providers, and businesses are experimenting with new layers and services built on top of the MCP. These emerging directions point toward a future where the MCP becomes a foundational layer of the web, much like HTTP or REST once were. Let's take a quick look at three initiatives that guide the next phase of the MCP.

12.6.1 MCP gateways

One problem today is LLM-powered web apps often need many separate integrations (calendars, email, commerce, and CRMs). Each integration is an independent MCP server with its own authorization, format, and state. When an agent tries to use many